

# CPT 205 Assessment 2

## A 3-Dimensional scene for futuristic cityscape

|                  |                                   |
|------------------|-----------------------------------|
| Module           | CPT205                            |
| Module Name      | Computer Graphics                 |
| Degree Programme | Information and Computing Science |
| Name             | Youming Yang                      |
| Student ID       | 2361217                           |

## 1. Design Description

This project is based on **C++** and **FreeGLUT**, and utilizes a series of **OpenGL** techniques such as the creation of geometry, hierarchical modelling, transformations, viewing and projection, lighting and materials, texture Mapping, animation and interactions have realized an interactive **3D** scene of a future city. It is divided into **six systems**, including the **ground background system** composed of water surfaces and trees, the **future city landmark system** consisting of the Eastern Gate, CB, and the Ferris wheel, the **floating sci-fi city** and **transportation system**. In the **robot system**, users can **freely move** and **transform the dynamic and static states of objects** by using the **third-person perspective** and interacting with the **keyboard and mouse**, to better observe the entire future city landscape. Moreover, the **sky system** further enriches the visual presentation through the **day-night cycle**. This report provides an overview of the **design and feature** of the future city, **user operation instructions**, and **scene screenshots**.

## 2. Design And Feature

### 2.1 Design

The original intention of this assignment was to create a future city and simulate the urban scene through 3d graphics. Therefore, this section provides an overview of the architectural concepts of future city and elaborates on the composition of the scenes.

#### Sky system

In the sky background, the sky part uses **gluSphere** in combination with textures to achieve the sky dome, and **glScalef** and **glRotatef** are used to surround and dynamically rotate the sky. The sun and moon generate sphere geometry by **glutSolidSphere** and calculate circular orbit coordinates by **trigonometric functions** with **glTranslatef** to realize orbital motion around the scene.

#### Ground background system

In the background system, the water surface is constructed by **glBegin(GL\_QUADS)** and texture coordinates, and then the **glTranslatef** and **glScalef** and wave parameters are used to realize the overall flow. The tree part adopts a billboard tree model with transparent textures and uses **glTranslatef**, **glRotatef**, and **glScalef** to achieve billboard transformation around the Y-axis towards the camera.

#### Future city landmark system

In the future urban landmark system, the Eastern Gate controls the inner half-width through a one-dimensional quadratic **Bezier function**, calculates the floor width and height layer by layer, and uses **wire frame technology**. **glutSolidCube+glTranslatef** and **glScalef** are stacked in the Y direction to form a

double-tower outline that gradually converges from bottom to top. IFC, on the other hand, explicitly calculates the vertex coordinates at each Angle using trigonometric functions, and concatenates the side walls, bottom surfaces, and the top elliptical covers inclined by cutDeg with **GL\_QUAD\_STRIP** and **GL\_TRIANGLE\_FAN**. The Ferris wheel uses a **hierarchical model** to build a structural framework through several **glutSolidCube** that have been **translated, rotated and scaled**, and then uses **polar coordinates** and **glRotatef** to arrange the unmanned aerial vehicle pods. In addition, a series of **GlutSolidSpheres** on the Ferris wheel are modulated in color through a **sine function** to create a flowing neon ring in the night scene, beautifying the overall environment. Finally, a wireframe building model relying on hexahedral wireframes, multi-layer **GL\_line\_loops** and semi-transparent fillers is formed through **glTranslatef** and repeated calls to create symmetrical building clusters. The CB building module forms inclined platforms by translating and rotating multiple **glutSolidCube** cubes around the Z-axis. The main body is composed of **GL\_QUADS** and textures

### Floating sci-fi city system

In the floating sci-fi city system, the system manually defines the quadrangular platform base, box-shaped tower and cylindrical side through **GL\_QUADS** and **GL\_QUAD\_STRIP**. And by establishing a local coordinate system and performing a series of **glTranslatef/glRotatef/glScalef** within it, the layout of the building and billboards is accomplished. The platform confirmed its position through **trigonometric functions**, while the orbiting monitor used **gluSphere** in conjunction with **trigonometric functions** to achieve circular orbit motion.

### Transportation system

In the transportation system, the car body, tires and roof glass are constructed by **glutSolidCube**, **glutSolidTorus** and **GL\_TRIANGLES**, and the layered flame is constructed by the superposition of **GL\_TRIANGLE\_FAN**. The train, on the other hand, has its body shape manually defined through **GL\_QUADS** and **In curve parameterization**, and its tail flame is composed of multiple scaled cubes. Both uniformly adopt **glTranslatef, glRotatef, glScalef** in combination with **glPushMatrix/glPopMatrix** for **hierarchical modeling**. The unmanned aerial vehicle also adopts **hierarchical modeling**, combining multiple levels of **glTranslatef** and **glRotatef** to form cubes and cylinders.

### Robot system

The robot system is drawn through **GL\_QUADS** and **gluSphere**. And use **glTranslatef, glRotatef, glScalef** in combination with **glPushMatrix/glPopMatrix** to establish **hierarchical joint structures**.

## 2.2 Feature

To make the scene above more realistic and beautiful, we have implemented specific graphics techniques using the OpenGL (FreeGLUT) library. The two techniques of **creation of geometry and transformations** have been elaborated in detail in 2.1.

### Viewing and projection

In terms of projection transformation, all six systems, except for the font in the lower left corner which uses the orthogonal projection **gluOrtho2D**, uniformly adopt the perspective projection based on **gluPerspective**, thereby ensuring that the entire picture has a consistent perspective effect and a sense of spatial depth. In terms of observing changes, the program uses **gluLookAt** to define a single camera view, and updates

**eyeX,Y,Z** as well as the gaze direction angles **eye\_rotation** and **eye\_updown** in real time through the keyboard and arrow keys. Realize **limited roaming** and perspective switching. It is worth mentioning that in the robot system, the spatial position and orientation of the robot model are **calculated by the eye/center parameters** of the camera, achieving a protagonist following effect like that **in a third-person perspective**.

## Lighting and materials

In terms of overall lighting, two parallel lights with diffuse reflection color and ambient light intensity - the sun (**GL\_LIGHT0**) and the moon (**GL\_LIGHT1**) - were set up through **glLightfv**, and they were activated through **glEnable(GL\_LIGHTING)** to provide lighting for the entire scene. In addition, the material properties of objects vary. The sun and the moon use **glMaterialfv** and **EMISSION** to make themselves glow, while the train locomotive and the buildings and countertops in the sky city system achieve a metallic texture with **glMaterialfv** and distribute **GL\_SHININESS** through **glMaterialfv**. To simulate the real response of the surface to constantly changing light sources.

## Texture mapping

In my scenario, texture mapping techniques are divided into **two categories**. The first category is to use **gluSphere** along with **gluQuadricTexture(obj, GL\_TRUE)** when drawing the sky and monitoring to **automatically enable the generation of quadric texture coordinates**. The second type is that when drawing the remaining textures, **glEnable(GL\_TEXTURE\_2D)** and **glBindTexture(GL\_TEXTURE\_2D, texture[])** images are mapped onto the geometry to enhance surface details. And align by manually specifying UV through **glTexCoord2f + glVertex3f**. Among them, many geometric blocks can also be repeatedly laid by setting **GL\_REPEAT** to form large ocean areas and exterior walls of houses.

## Hierarchical modelling

In this project, complex objects are represented **as a tree**, with each child node corresponding to a component (such as a drone on a Ferris wheel as a symphony, a drone's propeller, a train thruster, a car wheel, a robot arm), and a **local transformation** relative to the parent node is saved. When drawing, through **depth-first traversal** of this tree, combined with operations such as **glPushMatrix()/glPopMatrix()** and **glTranslatef, glRotatef, glScalef**, the transformation matrix is superimposed layer by layer along the path from the root node to the leaf node. In this way, when the upper-level parent node (such as the center of a Ferris wheel, the fuselage of a drone, the body of a train or a car, or the trunk) undergoes translation or rotation, all its child nodes will **automatically inherit these transformations**, thereby achieving the hierarchical motion effect.

## Animation

There is a total of **six different types** of animations driven by various drivers. 1. **Time-driven**: increment parameters (daytime, fw\_ang, wave) in **update()**, and then use **glRotatef/glTranslatef**, alpha to create animations such as water surface, sky, and Ferris wheel rotation. 2. **Angle parameter track**: Use **cos/sin( $\theta$ )** to place the automotive monitoring drone on a circular, annular, or figure-eight track, and then rotate it towards the tangential direction. 3. **Linear back-and-forth movement**: Trains and ground vehicles run along an X straight line and reverse direction at the boundary. 4. **Sinusoidal wave swing**: Robot joints, flame jitter, neon light color, perform periodic movements with **sin( $\omega t$ )**. 5. **Camera-driven**: The tree billboard is always facing the camera, and the robot follows in front of the camera. All calculations **rely on eye/center**.

## Billboard + $\alpha$ -channel texture technique

The forest section adopts **billboard technology**: by extracting the **right vector of the camera in the XZ plane** from the current model view matrix, a **Y-axis billboard rotation matrix** is constructed and applied to the texture quad of each tree, ensuring that the trees are always facing the camera directly. And **unlike normal texture maps**, this one converts the map into an **RGBA** format with **alpha** and changes the background color pixels to **alpha=0**. Then it is uploaded to the video memory via **glTexImage2D(GL\_TEXTURE\_2D, 0, GL\_RGBA, w, h, 0, GL\_RGBA, GL\_UNSIGNED\_BYTE, rgba)**. When drawing, use **glEnable(GL\_ALPHA\_TEST)**, **glAlphaFunc(GL\_GREATER, 0.1f)**, discard the very small fragment of alpha, and only retain the area of the trunk and crown that truly needs to be displayed. Combined with **glEnable(GL\_BLEND)** and **glBlendFunc(GL\_SRC\_ALPHA, GL\_ONE\_MINUS\_SRC\_ALPHA)**, a semi-transparent transition at the edges is achieved, presenting complex tree crown shapes in the picture.

### Alpha blending and transparency

The jet flame and tail flame do **not use** the traditional **glMaterial** fixed pipeline material. However, by turning off the lighting and enabling additive blending (**glBlendFunc(GL\_SRC\_ALPHA, GL\_ONE)**), the background will not be blocked, thereby generating a luminous flame effect with increased brightness as the overlap occurs, while using standard alpha blending (**glBlendFunc(GL\_SRC\_ALPHA, GL\_ONE\_MINUS\_SRC\_ALPHA)**) will cause the color to blend with the background to generate a soft and fading trajectory

### Wireframe

Future city is achieved through multi-channel rendering technology, where objects such as buildings are first drawn and then **wirefracted** using "**glutWireCube**". To avoid Z-fighting (flickering) caused by occupying almost the same depth, enable **glEnable(GL\_POLYGON\_OFFSET\_LINE)** before drawing the wireframe, and apply a slight offset towards the camera to the wireframe through **glPolygonOffset**. In this way, the visual upper box still adheres to the building surface, but the depth test will no longer conflict with the physical surface. In addition, the outline of the wireframe can also be directly drawn by **glPolygonMode(GL\_FRONT\_AND\_BACK, GL\_LINE)/glBegin(GL\_LINES)/GL\_LINE\_LOOP**

### Stencil-based CSG rendering

Through **inside (a, b, face)**, **glEnable(GL\_DEPTH\_TEST)**, **glColorMask(GL\_FALSE)**, **glStencilOp(GL\_INCR/GL\_DECR)**, and **glStencilFunc(GL\_ALWAYS)** **glStencilFunc(test, 0, 1)** extracts the visible part of an inside/outside b and then uses **glColorMask(GL\_FALSE)** in **fixup(a)**. Work with **glEnable(GL\_DEPTH\_TEST)** and **glDepthFunc(GL\_ALWAYS)** to correct the depth buffer to ensure the correct occlusion relationship of subsequent objects.

## 3. Instruction

### Keyboard Interaction

|                                       |                                              |                              |                            |
|---------------------------------------|----------------------------------------------|------------------------------|----------------------------|
| <b>1: Train start / stop</b>          | <b>2: Train show / hide</b>                  | <b>3: Cars start / stop</b>  | <b>4: Cars show / hide</b> |
| <b>5: Drones start / stop</b>         | <b>6: Drones show / hide</b>                 | <b>W / w: Look up</b>        | <b>S / s: Look down</b>    |
| <b>↑ : Move forward</b>               | <b>↓ : Move backward</b>                     | <b>← : Strafe left</b>       | <b>→ : Strafe right</b>    |
| <b>A / a: Rotate view to the left</b> | <b>D / d: Rotate view to the right</b>       | <b>Q / q: Move camera up</b> |                            |
| <b>E / e: Move camera down</b>        | <b>H / h: Toggle high-altitude view mode</b> |                              |                            |

### Mouse Interaction

|                                                      |                                              |
|------------------------------------------------------|----------------------------------------------|
| <b>Left mouse button: Robot walking start / stop</b> | <b>Right mouse button: Robot show / hide</b> |
|------------------------------------------------------|----------------------------------------------|

## 4. Screenshots play

